Network Reconnaissance Script Report

By : ABHISHEK BOMMISETTY

we are creating a python-based tool in networks which is used to identify any active host connected to it Should consist of two modes, they are:

Passive mode: This tool function is to listen the network traffic of a system passively and captures the data without sending the actual packet or any sort of signals might or might not be detected.

Active mode: This tool function is to actively send requests in this case, the ARP requests for detection and identification of the hosts that are in online this may be detected on the networks.

While running there are Arguments that we can see here are two

The network interface: This tells the tools on What network connection they should use. Each connection has its own unique identification. For example, enp0s3 and eth0 these are basically network interfaces for proper connection.

Mode indicator: So, this argument tells on What mode, should this tool operate in active mode or passive mode. The tool decides in What mode that it should draw.

We should also mention that while running the script we should give one of two command line **options -i or - - iface**. These options act as flags to tell the tool, what exactly the network interface should be used for scanning the operation.

The limited and restricted libraries that should be used in this module are **scapy, sys and os.**

Let us know what they are, and what they do…

scapy : This is a Python library, which is mainly used for discovering the network, capturing the packets and also manipulation of the packets. In this module, we are crafting and sending ARP packets using scapy library and captures the responses that we got.

sys: the system module is used for interacting with python environment for taking care of the command line arguments.

os : the operating system make sure tasks like running the system commands and having permissions to access the network interfaces of that operating system.

PART 1 - SETUP

Now we are setting up a net_recon.py file in the kali Linux using the nano command we enter the GNU command line. This is where we implement the python script. The script consists of the main function to set the network interface and creating a mode (active or passive) and a help function to run the script properly without any necessary arguments mainly used for user friendly functionality.

PART 2 – CODE EXPLANATION

I created a Python script called **net_recon.py** that serves as a network scanning tool using Scapy, a powerful library for handling network packets and discovery. This script supports two modes: passive and active scanning. It helps users detect active devices and analyse network traffic. My aim was to provide a straightforward and effective way to monitor network activity and identify connected devices. In passive mode, the tool listens to network traffic, while in active mode, it actively probes the network to find devices. This dual functionality offers flexibility for various situations and environments.

The **display_help()** function offers instructions on how to use the script, explaining the command-line arguments and options available. This is helpful because it guides users on running the script correctly, including the proper syntax and parameters. It includes arguments like -i for specifying the network interface and options like -p for passive mode and -a for active mode. These command-line arguments make the script adaptable, allowing it to work with different network interfaces and modes without hardcoding any settings. This makes it easier for anyone to use the tool according to their requirements.

The functions **print_table_header()** and **print_table()** are designed to display the results in a clear and organized manner. They create a dynamically updating table that shows information about detected hosts, such as their **IP and MAC addresses**. In passive mode, there's an additional column called "Host Activity" that indicates how frequently packets are detected from each host. In active mode, only the IP and MAC addresses are displayed. These functions ensure that the output is easy to read and understand, and the dynamic updating keeps the information current without overwhelming the user, especially during real-time monitoring.

In passive mode, the **passive_scan()** function uses Scapy's **sniff()** function to listen for ARP packets on the specified network interface. ARP packets are ideal for detecting devices on a local network because they reveal both IP and MAC addresses. Each ARP packet is processed to extract the source MAC and IP addresses, and this information is stored in a dictionary for each unique host. By counting the number of times a host's ARP packets are detected, we can monitor the activity level of each device, which might provide insights into network behaviour. The command os.system('clear') is used to clear the screen each time new data is printed, keeping the output clean and up-to-date.

The **active_scan()** function discovers devices on the network by sending ARP requests to a specified IP range, allowing it to identify active hosts. Scapy's **srp()** function is used to send packets and collect responses, enabling efficient, low-level interaction with the network. This function targets IP addresses in the range **"192.168.1.0/24,"** which is a common private IP address range for local networks, but this can be adjusted as needed. By sending ARP requests, we can detect active hosts

based on their replies, since any live device on the network will respond with its IP and MAC addresses. The active scanning mode is useful when we want a quick inventory of active devices on the network without relying on passive traffic.

PART 3 – LOG OF CHANGES MADE

DATE	CHANGES MADE	REASON
02-11-2024	Read the class PPTs and labs and I started creating the basic structure of the code	Very basic indentation errors near print commands
03-11-2024	I then started writing the code for the passive_scan functions in the code	
05-112024	Started coding on active_scan	Found some errors in the os. System part of the code
05-11-2024	Started documenting writing what I understood from the coding	
07-11-2024	Added the print_table and its header functions.	Modification of some pervious code changes and mistakes
08-11-2024	Tested both passive mode with the give ARP script and tested active code using the Wireshark	Faced no output errors if we run it in other interfaces such as eth1 and others I got some name errors
09-11-2024	Added the comments to the coding part and had done some documenting	
10-11-2024	Tested the code again and ran both modes and confirmed the output matches my screenshot's and finalised the documentation.	

PART 5 – FINAL OUTPUT

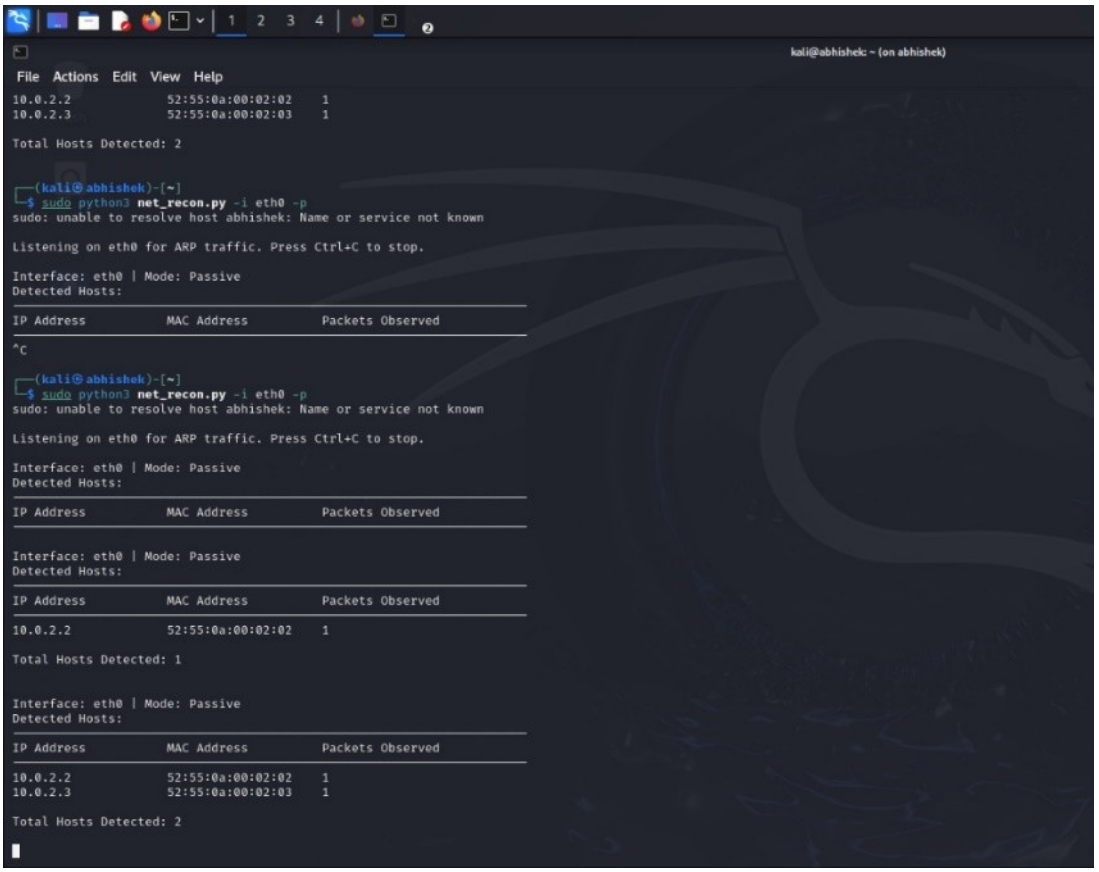

Fig 1.1

In the above figure 1.1 we can see that the ARP traffic in passive mode yet active must be stared just checking what the interface and display looks like on eth0 interface.

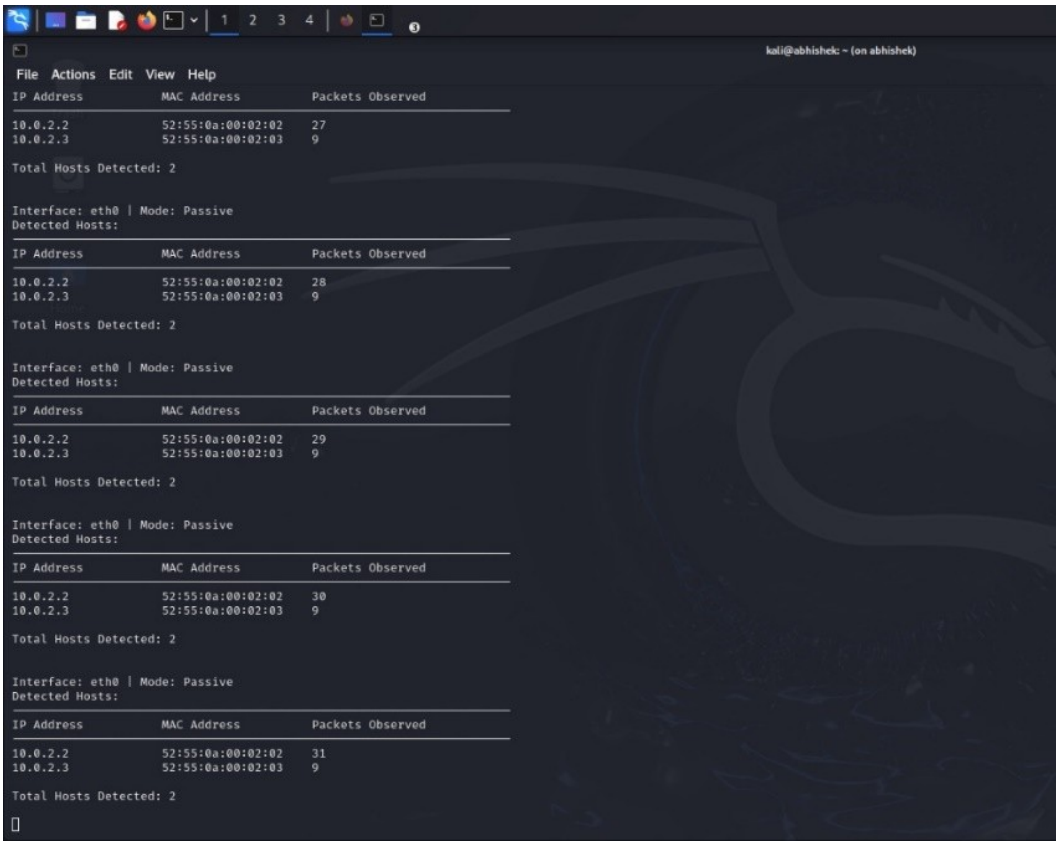

Fig 1.2

In the above figure 1.2 it shows the hosts IP address and MAC addresses the packet count is increasing with 2 hosts detected in the interface.

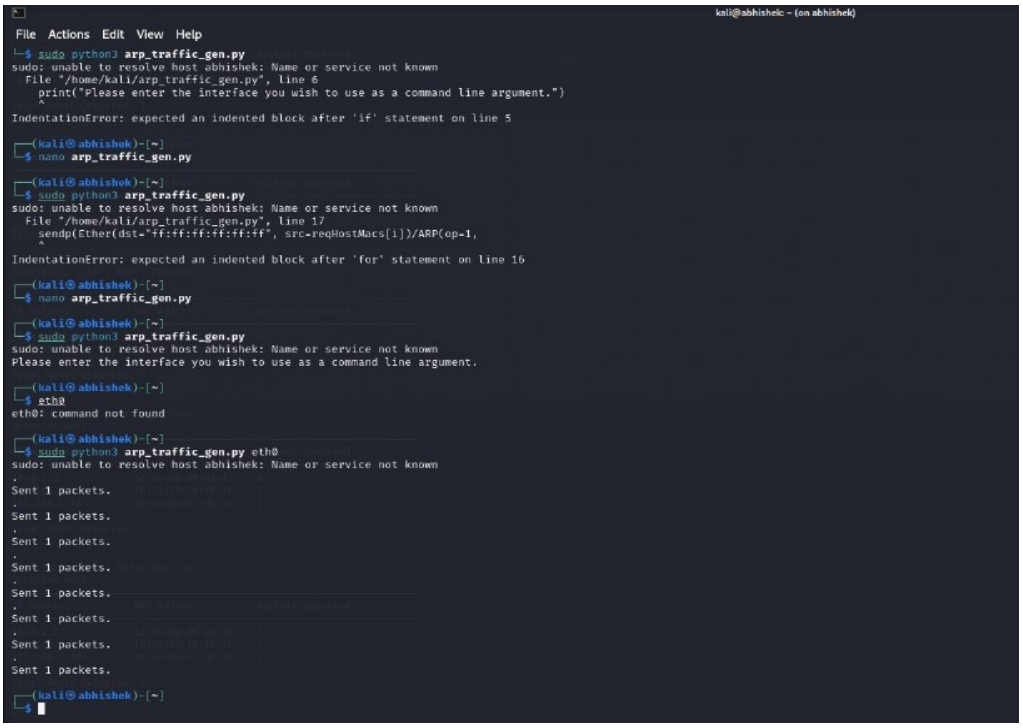

Fig 1.3

In the above figure 1.3 we are using the given script to the tool to run in passive mode. So, we are simulating the network traffic activity for net_recon.py to detect.

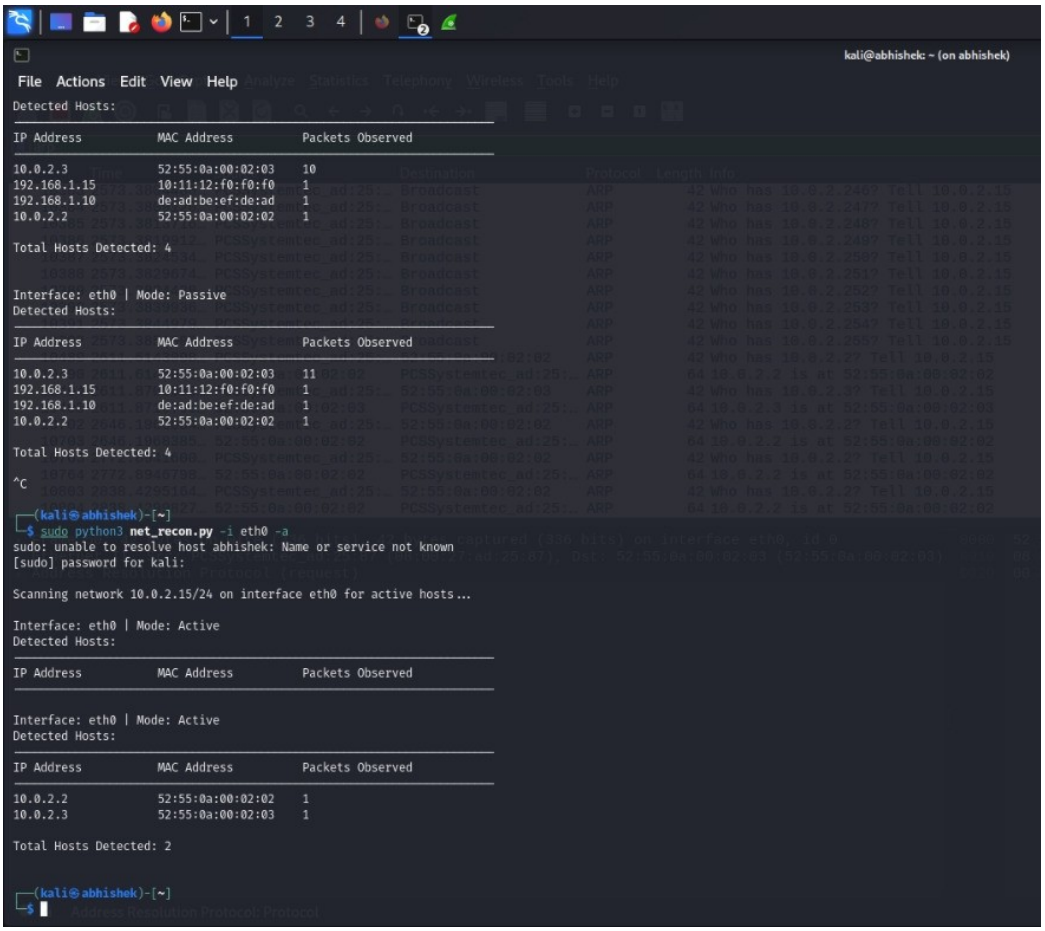

Fig 1.4

In the above figure we can see some IP and MAC addresses has been added.

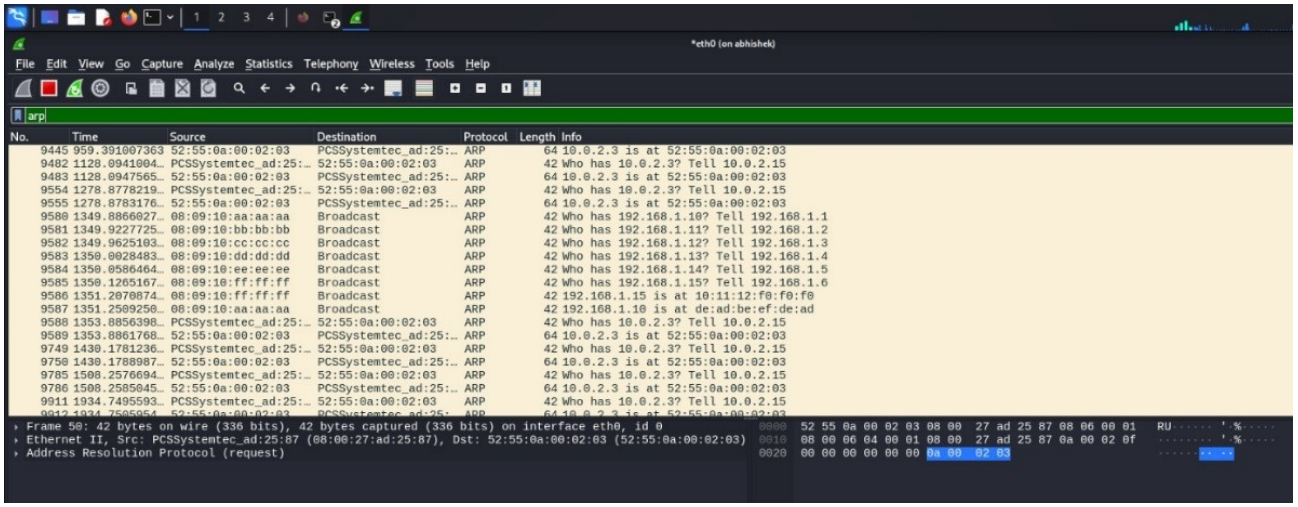

Fig 1.5

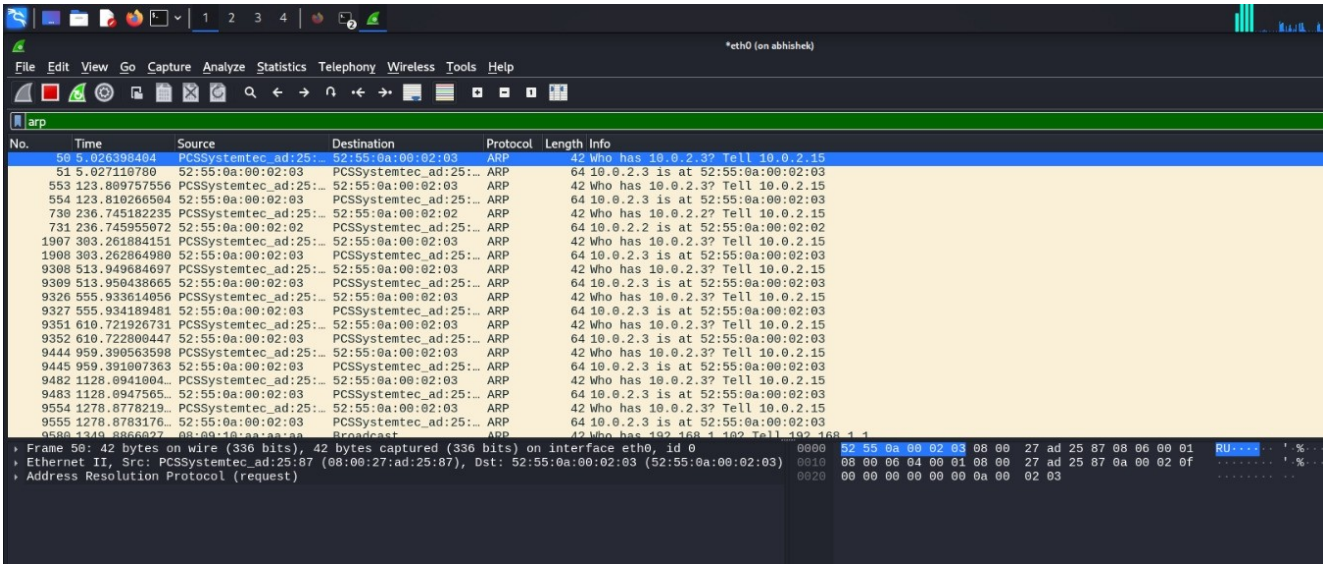

Fig 1.6

In the above figures (1.5 & 1.6) we can see in Wireshark that the ARP traffic is being captured communicating who Is who and requesting the IP and MAC across the network.